

Git 协作开发规范（短期功能分支）

本文档用于规范团队 Git 协作流程，确保多人开发时版本管理一致、清晰、可追踪。

分支结构

master（稳定分支）

- 始终指向 **生产环境已发布版本**
- 一般禁止直接 commit
- 仅通过 release 或 hotfix 合并更新

develop（集成分支）

- 所有功能最终都会先合并到 develop
- 不允许直接 push

feature/*（功能分支）

- 用于开发新功能
- 创建自 develop，完成后合并回 develop

release/*（发布准备分支）

- 发布前用于测试、文档和版本号调整
- 创建自 develop，最终分别合并到 master 和 develop

hotfix/*（线上紧急修复）

- 从 master 创建
- 修复完成后合并回 master 与 develop

```
1  master ← (release/hotfix)
2      ↑
3  develop ← (feature/*)
```

分支保护规则

为确保历史清晰、流程规范，应在平台开启保护规则防止误操作：

- **master 禁止直接 push**
- **develop 禁止直接 push**
- feature 分支允许 push
- 所有合并必须通过 **Pull Request**
- 至少需要 **1 名 reviewer 审核通过**
- master 合并必须确保 **CI 通过**

工作流 (Workflow)

初始设置

```
1  git clone <仓库URL>
2  cd <仓库名>
3
4  git checkout develop
5  git pull origin develop
```

开始新功能开发

```
1  git checkout develop
2  git pull origin develop
3
4  git checkout -b feature/功能描述
```

示例:

```
1  git checkout -b feature/user-login
2  git checkout -b feature/add-payment
```

开发与提交

```
1  git add .
2  git commit -m "feat: 功能描述"
3
4  # 后续持续提交
5  git commit -m "feat: 完善 xxx 功能"
6  git commit -m "fix: 修复 xxx 问题"
7
8  git push -u origin feature/功能描述
```

完成功能并合并到 develop

```
1  git checkout feature/功能描述
2  git pull origin feature/功能描述
```

通过 Pull Request 合并:

- 开发者创建 PR → 合并至 develop
- Reviewer 审核通过后合并
- CI 必须通过

合并方式:

统一使用 Merge commit (no-ff) 合并方式, 不允许 squash merge / rebase merge, 除非团队已有明确规定

```
1  git merge --no-ff feature/功能描述
```

合并完成后清理：

```
1 git branch -d feature/功能描述
2 git push origin --delete feature/功能描述
```

发布流程 (release)

release 分支 **只能从 develop 创建**，不能从 feature 或 hotfix 创建。

发布流程：develop → release → master

创建发布分支

```
1 git checkout develop
2 git pull origin develop
3
4 git checkout -b release/v1.x.x
5 git push -u origin release/v1.x.x
```

发布测试阶段（版本号更新、文档、最终检查）

```
1 git add .
2 git commit -m "chore: 准备发布 v1.x.x"
3 git push
```

合并到 master（正式发布）

```
1 git checkout master
2 git pull origin master
3 git merge --no-ff release/v1.x.x
```

打标签并推送：

```
1 git tag -a v1.x.x -m "版本 v1.x.x"
2 git push origin master
3 git push origin v1.x.x
```

同步 develop

```
1 git checkout develop
2 git merge --no-ff release/v1.x.x -m "merge release/v1.x.x"
3 git push origin develop
```

删除 release 分支

```
1 git branch -d release/v1.x.x
2 git push origin --delete release/v1.x.x
```

紧急修复流程 (hotfix)

创建 hotfix 分支

```
1 git checkout master
2 git pull origin master
3
4 git checkout -b hotfix/问题描述
5 git push -u origin hotfix/问题描述
```

修复与提交

```
1 git add .
2 git commit -m "fix: 紧急修复"
3 git push
```

合并到 master 和 develop

```
1 # master
2 git checkout master
3 git merge --no-ff hotfix/问题描述 -m "merge hotfix/问题描述"
4 git push
5
6 # develop
7 git checkout develop
8 git merge --no-ff hotfix/问题描述 -m "merge hotfix/问题描述"
9 git push
```

删除 hotfix 分支

```
1 git branch -d hotfix/问题描述
2 git push origin --delete hotfix/问题描述
```

回滚策略 (Rollback Strategy)

所有公共分支 (master、develop) 禁止使用 `git reset` 回滚。

统一使用安全的 `git revert`。

回滚 master 的发布版本

```
1 git checkout master
2 git pull origin master
3
4 # 回滚单个提交
5 git revert <commit-hash>
6
7 # 回滚 merge commit
8 git revert -m 1 <merge-commit-hash>
9
10 git push origin master
```

同步 develop:

```
1 git checkout develop
2 git pull origin develop
3 git merge master
4 git push
```

回滚 develop

```
1 git checkout develop
2 git pull origin develop
3 git revert <commit-hash> 或 merge-hash>
4 git push
```

回滚 release（未发布）

release 分支还未进入 master，可直接 revert 或直接删除：

确保 release 尚未向 master 或 develop 合并，否则不要删除。

```
1 git branch -D release/v1.x.x
2 git push origin --delete release/v1.x.x
```

然后重新从 develop 创建 release。

回滚 hotfix

已合并 hotfix:

```
1 git checkout master
2 git revert <commit-hash>
3 git push
4
5 git checkout develop
6 git revert <commit-hash>
7 git push
```

分支命名规范

- `feature/xxx` — 新功能
- `fix/xxx` — 一般 Bug 修复
- `hotfix/xxx` — 线上紧急修复
- `release/x.y.z` — 发布分支

提交信息规范 (Commit Message)

```
1 feat: 新功能
2 fix: 修复问题
3 docs: 文档更新
4 style: 代码风格 (无逻辑改动)
5 refactor: 代码重构 (无功能变化)
6 test: 测试相关
7 chore: 构建流程 / 工具相关
```

常用命令

Git 常用命令

```
1 # 查看本地与远程分支列表，并显示每个分支的最后一次提交信息 (-vv 显示详细信息)
2 git branch -avv
3
4 # 以图形方式查看所有分支的提交历史 (结构化展示分支与合并关系)
5 git log --oneline --graph --all
6
7 # 查看当前工作区与暂存区的状态 (包括未跟踪文件、修改内容)
8 git status
```

回滚相关命令

```
1 # 将 HEAD 回退到前一个提交，但保留工作区与暂存区的文件 (不丢失代码)
2 # 禁止在公共分支使用 (master / develop)
3 # 仅可用于本地分支 (例如 feature)
4 git reset --soft HEAD^
5
6 # 放弃当前工作区中文件的修改，将文件恢复到最近一次提交的状态 (谨慎操作)
7 git checkout -- <文件>
```

代码审查 (Code Review)

1. 开发者在 feature/ 分支完功能后创建 PR
2. Reviewer 审阅代码
3. CI 通过后允许合并
4. 合并完成后删除功能分支